Отчёт по лабораторной работе №2

Управление версиями

Кирилл Исаев

Содержание

1 Цель работы 5

2 Выполнение лабораторной работы 6

3 Вывод 17

4 Контрольные вопросы 18

Список иллюстраций

2.1 Загрузка пакетов 7
2.2 Параметры репозитория 8
2.3 rsa-4096 9
2.4 ed25519 10
2.5 GPG ключ 11
2.6 GPG ключ 12
2.7 Параметры репозитория 13
2.8 Связь репозитория с аккаунтом 14
2.9 Загрузка шаблона 15
2.10 Первый коммит 16

Список таблиц

1 Цель работы

Целью данной работы является изучение идеологии и применения средств контроля версий и освоение умений работать с git.

2 Выполнение лабораторной работы

Устанавливаем git, git-flow и gh.

```
keisaev@keisaev:~$ git
использование: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
           [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
           [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--no-lazy-fetch]
           [--no-optional-locks] [--no-advice] [--bare] [--git-dir=<path>]
           [--work-tree=<path>] [--namespace=<name>] [--config-env=<name>=<envvar>]
           <command> [<args>]

Стандартные команды Git используемые в различных ситуациях:

создание рабочей области (смотрите также: git help tutorial)
   clone     Клонирование репозитория в новый каталог
   init      Создание пустого репозитория Git или переинициализация существующего

работа с текущими изменениями (смотрите также: git help everyday)
   add       Добавление содержимого файла в индекс
   mv        Перемещение или переименование файла, каталога или символьной ссылки
   restore   Восстановление файлов в рабочем каталоге
   rm        Удаление файлов из рабочего каталога и индекса

просмотр истории и текущего состояния (смотрите также: git help revisions)
   bisect    Выполнение двоичного поиска коммита, который вносит ошибку
   diff      Вывод разницы между коммитами, коммитом и рабочим каталогом и т.д.
   grep      Вывод строк, соответствующих шаблону
   log       Вывод истории коммитов
   show      Вывод различных типов объектов
   status    Вывод состояния рабочего каталога

выращивание, маркировка и правка вашей общей истории
   branch    Вывод списка, создание или удаление веток
   commit    Запись изменений в репозиторий
   merge     Объединение одной или нескольких историй разработки вместе
   rebase    Повторное применение коммитов над верхушкой другой ветки
   reset     Сброс текущего состояния HEAD на указанное состояние
   switch    Переключение веток
   tag       Создание, вывод списка, удаление или проверка метки, подписанной с помощью GPG

совместная работа (смотрите также: git help workflows)
   fetch     Загрузка объектов и ссылок из другого репозитория
   pull      Извлечение изменений и объединение с другим репозиторием или локальной веткой
   push      Обновление внешних ссылок и связанных объектов
```

Рис. 2.1: Загрузка пакетов

Зададим имя и email владельца репозитория, кодировку и прочие параметры.

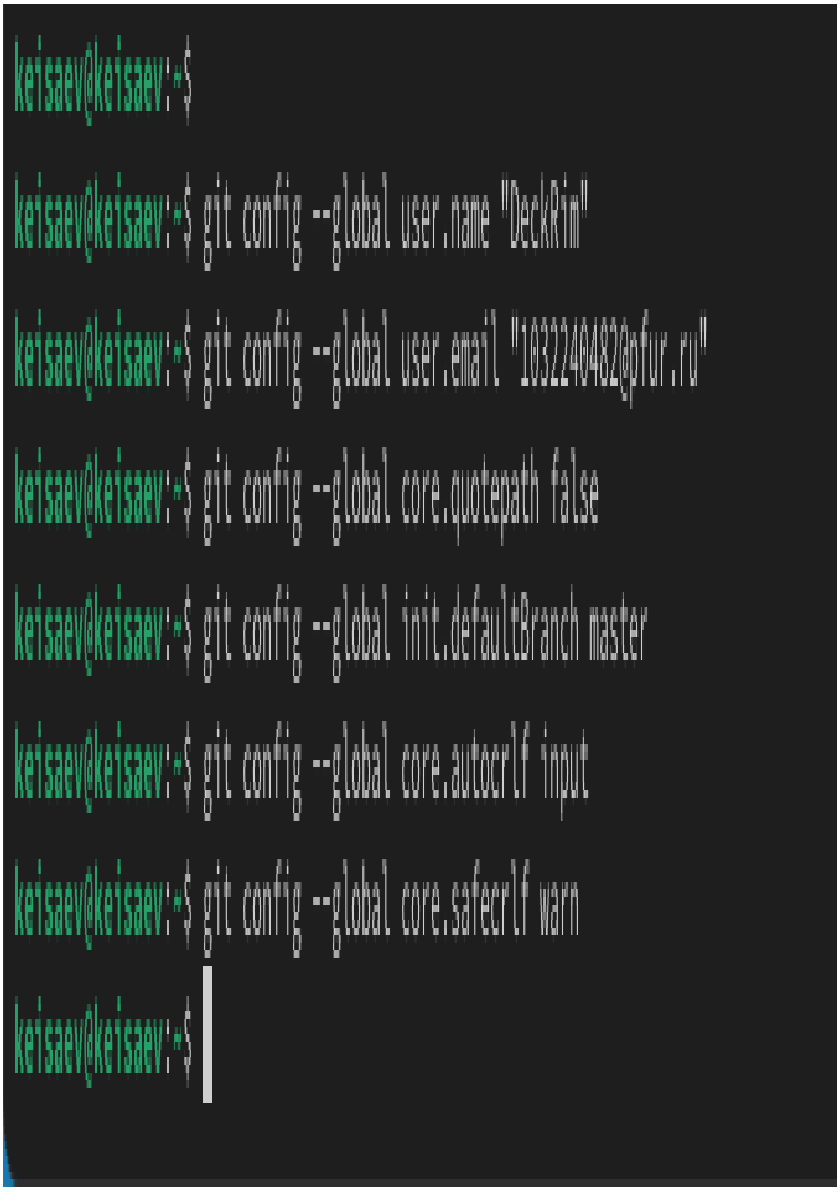

Рис. 2.2: Параметры репозитория

Создаем SSH ключи

```
keisaev@keisaev:~$
keisaev@keisaev:~$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/keisaev/.ssh/id_rsa):
Created directory '/home/keisaev/.ssh'.
Enter passphrase for "/home/keisaev/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/keisaev/.ssh/id_rsa
Your public key has been saved in /home/keisaev/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:AgJlykK2avx6gGdJuVoBoIIxCKDlKsLhLGISXAiByy0 keisaev@keisaev
The key's randomart image is:
+---[RSA 4096]----+
|^==.             |
|%Xo              |
|XB+..            |
|XE=o .           |
|XO.+  . S        |
|X.B    .         |
| =..             |
|. ..             |
| ..              |
+----[SHA256]-----+
keisaev@keisaev:~$
```

Рис. 2.3: rsa-4096

```
keisaev@keisaev:~$ ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/keisaev/.ssh/id_ed25519):
Enter passphrase for "/home/keisaev/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/keisaev/.ssh/id_ed25519
Your public key has been saved in /home/keisaev/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:8R27l86GagkT3ZTCrrpGRdLSu6yN/CkmMR/pxZ+HQqw keisaev@keisaev
The key's randomart image is:
+--[ED25519 256]--+
|      o .   .    |
|     o + o o     |
|      +.+ +.     |
|       +oo..o    |
|      *S+. o     |
|   o + @    . .  |
|    B X + +..o   |
|   . E o.* o+.   |
|    +.+oo.o .o   |
+----[SHA256]-----+
keisaev@keisaev:~$
```

Рис. 2.4: ed25519

Создаем GPG ключ

```
Ваш выбор? 1
длина ключей RSA может быть от 1024 до 4096.
Какой размер ключа Вам необходим? (3072) 4096
Запрошенный размер ключа - 4096 бит
Выберите срок действия ключа.
        0 = не ограничен
      <n>  = срок действия ключа - n дней
      <n>w = срок действия ключа - n недель
      <n>m = срок действия ключа - n месяцев
      <n>y = срок действия ключа - n лет
Срок действия ключа? (0) 0
Срок действия ключа не ограничен
Все верно? (y/N) y

GnuPG должен составить идентификатор пользователя для идентификации ключа.

Ваше полное имя: DeckRim
Адрес электронной почты: 1032240482@pfur.ru
Примечание:
Вы выбрали следующий идентификатор пользователя:
    "DeckRim <1032240482@pfur.ru>"

Сменить (N)Имя, (C)Примечание, (E)Адрес; (O)Принять/(Q)Выход? O
Необходимо получить много случайных чисел. Желательно, чтобы Вы
в процессе генерации выполняли какие-то другие действия (печать
на клавиатуре, движения мыши, обращения к дискам); это даст генератору
случайных чисел больше возможностей получить достаточное количество энтропии.
Необходимо получить много случайных чисел. Желательно, чтобы Вы
в процессе генерации выполняли какие-то другие действия (печать
на клавиатуре, движения мыши, обращения к дискам); это даст генератору
случайных чисел больше возможностей получить достаточное количество энтропии.
gpg: /home/keisaev/.gnupg/trustdb.gpg: создана таблица доверия
gpg: создан каталог '/home/keisaev/.gnupg/openpgp-revocs.d'
gpg: сертификат отзыва записан в '/home/keisaev/.gnupg/openpgp-revocs.d/78E07B2FAAF8A4E43C97E34FC256F576006A4852.rev'.
открытый и секретный ключи созданы и подписаны.

pub   rsa4096 2025-09-09 [SC]
      78E07B2FAAF8A4E43C97E34FC256F576006A4852
uid                      DeckRim <1032240482@pfur.ru>
sub   rsa4096 2025-09-09 [E]
```

Рис. 2.5: GPG ключ

Добавляем GPG ключ в аккаунт

Рис. 2.6: GPG ключ

Настройка автоматических подписей коммитов git

```
keisaev@keisaev:~$ gpg --list-secret-keys --keyid-format LONG
gpg: проверка таблицы доверия
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: глубина: 0  достоверных:  1  подписанных:  0  доверие: 0-, 0q, 0n, 0m, 0f, 1u
[keyboxd]
---------
sec   rsa4096/C256F576006A4852 2025-09-09 [SC]
      78E07B2FAAF8A4E43C97E34FC256F576006A4852
uid                 [ абсолютно ] DeckRim <1032240482@pfur.ru>
ssb   rsa4096/F64D7994163F562E 2025-09-09 [E]

keisaev@keisaev:~$ gpg --armor --export C256F576006A4852 | xclip -sel clip
keisaev@keisaev:~$ git config --global user.signingkey C256F576006A4852
keisaev@keisaev:~$ git config --global commit.gpgsign true
keisaev@keisaev:~$ git config --global gpg.program $(which gpg2)
keisaev@keisaev:~$
```

Рис. 2.7: Параметры репозитория

Настройка gh

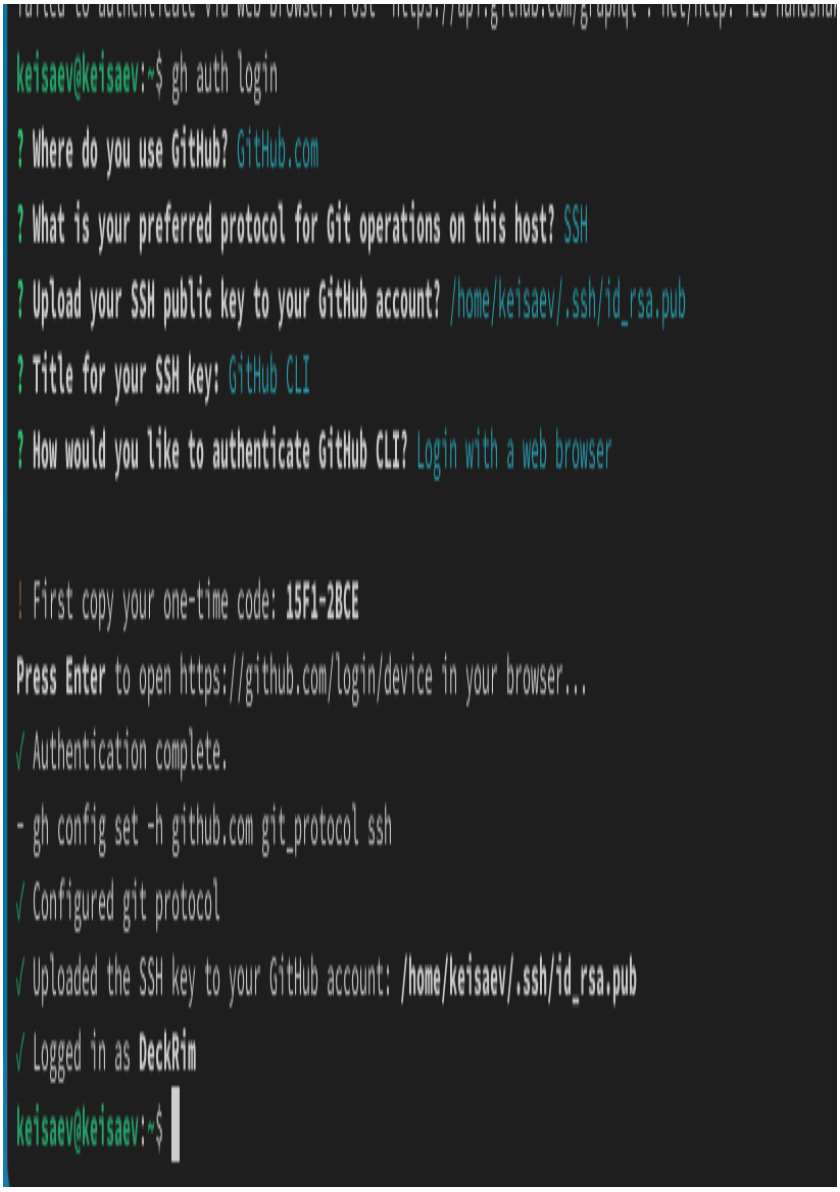

Рис. 2.8: Связь репозитория с аккаунтом

Загрузка шаблона репозитория и синхронизация

```
keisaev@keisaev:~$ mkdir -p ~/work/study/2024-2025/"Операционные системы"
keisaev@keisaev:~$ cd ~/work/study/2024-2025/"Операционные системы"
keisaev@keisaev:~/work/study/2024-2025/Операционные системы$ gh repo create os-intro --template=yamadharma/course-directo
ry-student-template --public
✓ Created repository DeckRim/os-intro on GitHub
  https://github.com/DeckRim/os-intro
keisaev@keisaev:~/work/study/2024-2025/Операционные системы$ git clone --recursive git@github.com:DeckRim/os-intro.git os
-intro
Клонирование в «os-intro»...
The authenticity of host 'github.com (140.82.121.3)' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
remote: Enumerating objects: 38, done.
remote: Counting objects: 100% (38/38), done.
remote: Compressing objects: 100% (36/36), done.
remote: Total 38 (delta 1), reused 27 (delta 1), pack-reused 0 (from 0)
Получение объектов: 100% (38/38), 23.46 КиБ | 272.00 КиБ/с, готово.
Определение изменений: 100% (1/1), готово.
Подмодуль «template/presentation» (https://github.com/yamadharma/academic-presentation-markdown-template.git) зарегистрир
ован по пути «template/presentation»
Подмодуль «template/report» (https://github.com/yamadharma/academic-laboratory-report-template.git) зарегистрирован по пу
ти «template/report»
Клонирование в «/home/keisaev/work/study/2024-2025/Операционные системы/os-intro/template/presentation»...
```

Рис. 2.9: Загрузка шаблона

Подготовка репозитория и коммит изменений

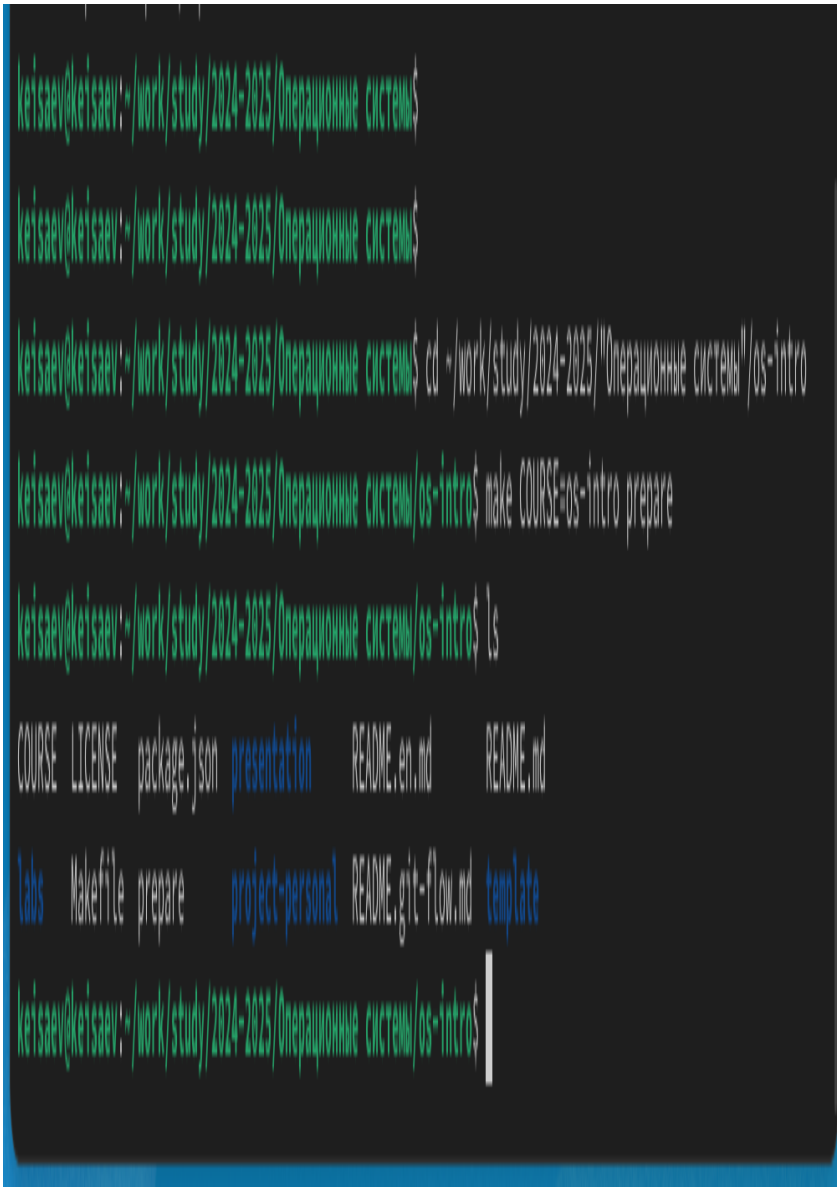

Рис. 2.10: Первый коммит

3 Вывод

Мы приобрели практические навыки работы с сервисом github.

4 Контрольные вопросы

1. Что такое системы контроля версий (VCS) и для решения каких задач они предназначаются?

Системы контроля версий (Version Control System, VCS) применяются при работе нескольких человек над одним проектом. Обычно основное дерево проекта хранится в локальном или удалённом репозитории, к которому настроен доступ для участников проекта. При внесении изменений в содержание проекта система контроля версий позволяет их фиксировать, совмещать изменения, произведённые разными участниками проекта, производить откат к любой более ранней версии проекта, если это требуется

2. Объясните следующие понятия VCS и их отношения: хранилище, commit, история, рабочая копия.

- хранилище - пространство на накопителе где расположен репозиторий
- commit - сохранение состояния хранилища
- история - список изменений хранилища (коммитов)
- рабочая копия - локальная копия сетевого репозитория, в которой работает программист. Текущее состояние файлов проекта, основанное на версии, загруженной из хранилища (обычно на последней)

3. Что представляют собой и чем отличаются централизованные и децентрализованные VCS? Приведите примеры VCS каждого вида.

Централизованные системы контроля версий представляют собой приложения типа клиент-сервер, когда репозиторий проекта существует в единственном экземпляре и хранится на сервере. Доступ к нему осуществлялся через специальное клиентское приложение. В качестве примеров таких программных продуктов можно привести CVS, Subversion.

Распределенные системы контроля версий (Distributed Version Control System, DVCS) позволяют хранить репозиторий (его копию) у каждого разработчика, работающего с данной системой. При этом можно выделить центральный репозиторий (условно), в который будут отправляться изменения из локальных и, с ним же эти локальные репозитории будут синхронизироваться. При работе с такой системой, пользователи периодически синхронизируют свои локальные репозитории с центральным и работают непосредственно со своей локальной копией. После внесения достаточного количества изменений в локальную копию они (изменения) отправляются на сервер. При этом сервер, чаще всего, выбирается условно, т.к. в большинстве DVCS нет такого понятия как “выделенный сервер с центральным репозиторием”.

4. Опишите действия с VCS при единоличной работе с хранилищем.

Один пользователь работает над проектом и по мере необходимости делает коммиты, сохраняя определенные этапы.

5. Опишите порядок работы с общим хранилищем VCS.

Несколько пользователей работают каждый над своей частью проекта. При этом каждый должен работать в своей ветки. При завершении работы ветка пользователя сливается с основной веткой проекта.

6. Каковы основные задачи, решаемые инструментальным средством git?

- Ведение истории версий проекта: журнал (log), метки (tags), ветвления (branches).

- Работа с изменениями: выявление (diff), слияние (patch, merge).
- Обеспечение совместной работы: получение версии с сервера, загрузка обновлений на сервер.

7. Назовите и дайте краткую характеристику командам git.

- git config - установка параметров
- git status - полный список изменений файлов, ожидающих коммита
- git add . - сделать все измененные файлы готовыми для коммита.
- git commit -m “[descriptive message]” - записать изменения с заданным сообщением.
- git branch - список всех локальных веток в текущей директории.
- git checkout [branch-name] - переключиться на указанную ветку и обновить рабочую директорию.
- git merge [branch] — соединить изменения в текущей ветке с изменениями из заданной.
- git push - запушить текущую ветку в удаленную ветку.
- git pull - загрузить историю и изменения удаленной ветки и произвести слияние с текущей веткой.

8. Приведите примеры использования при работе с локальным и удалённым репозиториями.

- git remote add [имя] [url] — добавляет удалённый репозиторий с заданным именем;
- git remote remove [имя] — удаляет удалённый репозиторий с заданным именем;
- git remote rename [старое имя] [новое имя] — переименовывает удалённый репозиторий;
- git remote set-url [имя] [url] — присваивает репозиторию с именем новый адрес;

- git remote show [имя] — показывает информацию о репозитории.

9. Что такое и зачем могут быть нужны ветви (branches)?

Ветвление — это возможность работать над разными версиями проекта: вместо одного списка с упорядоченными коммитами история будет расходиться в определённых точках. Каждая ветвь содержит легковесный указатель HEAD на последний коммит, что позволяет без лишних затрат создать много веток. Ветка по умолчанию называется master, но лучше назвать её в соответствии с разрабатываемой в ней функциональностью.

10. Как и зачем можно игнорировать некоторые файлы при commit?

Зачастую нам не нужно, чтобы Git отслеживал все файлы в репозитории, потому что в их число могут входить: